ScratchBird Query Optimization – Issues & Gaps Report

Scope: This report summarizes issues found by comparing the current ScratchBird source code against (1) the original Query Optimizer Specification and (2) the RLS-for-SELECT / Vector Executor report. Focus is on correctness, security invariants, and optimization completeness.

Executive Summary

- The optimizer module is real and non-trivial (planner, join ordering, stats, selectivity, cost model, matchers/rewrites).
- Several spec-level optimizer features appear incomplete or absent (extended stats, plan cache, adaptive execution, parallel planning/costing, rich EXPLAIN).
- The highest-risk issue is lack of an explicit optimizer-level security barrier for RLS, which can enable accidental policy bypass when adding pushdown, index-only paths, LIMIT short-circuiting, or parallel execution.
- Current code likely behaves safely for many queries today, but the system is in a 'hazard zone': adding more aggressive optimizations without explicit RLS-aware plan constraints can create silent security bugs.

Observed Implemented Foundations

- Cost-based planning scaffolding: Query planner, cost model, selectivity estimator, and statistics manager modules exist and are integrated.
- Join ordering infrastructure exists (at minimum DP-style ordering for smaller join graphs; larger-graph heuristics require verification).
- Rewrite/matching infrastructure exists (expression/predicate matchers, common subexpression elimination, MV rewrite hooks).
- Execution layer exists (SBLR) with a pathway for vectorization and an execution plan structure to target.

Issue Catalog (Prioritized)

ID	Severity	Issue	Impact
OPT-RLS-01	Critical	Optimizer lacks an explicit RLS security barrier operator / pushdown	Future optimizations (pushdown / index access / LIMIT / parallelism can
OPT-STAT-01	High	Extended statistics (multi-column/ND histograms/FDs, ndistinct, MCV)	Join order and selectivity estimation degrade; plans unstab
OPT-CACHE-01	High	Plan cache lifecycle (keying/invalidation/LRU/TTL) not implemented as specified	Repeated query workloads may replan too often; correctn
OPT-ADAPT-01	Medium	Adaptive execution & runtime feedback/replanning	Large queries with misestimates suffer; risk of timeouts/ou

OPT-PAR-01	Medium	Parallel planning/costing not evident at spec level	Parallel execution cannot be chosen safely/optimally; risk
OPT-EXPL-01	Medium	EXPLAIN/ANALYZE output contract not clearly implemented	Hard to debug and validate plans; slows performance tu
OPT-PUSH-01	Medium	Predicate pushdown rules not proven to be constrained by security barriers	Even with RLS, could break semantics around volatile f
OPT-OP-01	Low	Operator coverage and cost calibration incomplete	Edge-case queries may pick pathological plans.

Critical Detail: RLS & Optimization Interaction

The RLS report requires that row-level security be enforced as a non-bypassable stage between MGA visibility and any user-visible output. Optimizer transformations must never move user predicates, join reorders, index access paths, LIMIT, or parallelism in a way that filters rows before RLS policy evaluation. Without an explicit plan-node barrier (e.g., RLSFilter) plus rewrite rules preventing reordering across it, the system is vulnerable to accidental policy bypass as optimization sophistication increases.

- If predicate pushdown is enabled: user WHERE predicates must not be pushed below the RLS barrier.
- If index-only/late materialization is used: required policy columns must be fetched before RLS evaluation.
- If LIMIT is pushed down or early-terminated: it must apply after RLS filtering (or implement 'secure LIMIT' semantics).
- If parallel scan/filter is used: RLS must be evaluated in each worker with the correct session context and policy bindings.

Recommended Acceptance Tests (Minimum)

- RLS non-bypass tests: same query with different user contexts must never return rows outside policy (including with LIMIT, ORDER BY, indexes).
- Predicate pushdown safety tests: volatile functions, outer joins, NULL-sensitive predicates, and RLS policies in combination.
- Planner stability tests: skewed data sets and correlated predicates should not produce catastrophic plans.
- Explainability: EXPLAIN shows RLS barrier placement and indicates which predicates were pushed where (or explicitly not pushed).

Conclusion

ScratchBird’s optimizer is structurally present and capable, but it is currently missing several spec-level capabilities. The most urgent gap is formalizing RLS as an optimizer-visible security barrier and enforcing transformation constraints. Once RLS constraints are encoded into plan construction and rewrite rules, the remaining optimization gaps are largely performance and operability improvements that can be implemented incrementally.